

User stories:

pdf

Here are my 5 user stories:

The five:

AI suggestion 3: the job seeker can provide the career goals and their CV to be able to match the user with the most appropriate jobs.

filter job postings (by location, title, or category) so that I can quickly find roles relevant to me.

As a job seeker, I want to save jobs to a favourites list so that I can revisit postings I'm interested in later.

As a job seeker, I want to create an account and log in to access my profile and View application history.

recruiters can view my qualifications when I apply to jobs.

Create me 10 more users stories

Drafting additional user stories for a CareerConnect project.

Drafting additional user stories for a CareerConnect project.

Here are 10 additional user stories for CareerConnect, covering areas your current five don't touch yet — resume management, recruiter-side features, application tracking, and notifications:

Job Seeker stories

1. As a job seeker, I want to upload and update my resume so that recruiters can view my latest qualifications.
2. As a job seeker, I want to track the status of each application (Applied, Interview, Offered, Rejected) so that I can stay organized throughout my job search.
3. As a job seeker, I want to receive notifications when a job I'm interested in is about to close so that I don't miss application deadlines.

4. As a job seeker, I want to view a dashboard summarizing all my past and current applications so that I can quickly assess my progress.
5. As a job seeker, I want to receive AI-generated feedback on my resume so that I can improve it before applying.

Recruiter stories

6. As a recruiter, I want to create and publish job postings so that job seekers can discover and apply to open roles.
7. As a recruiter, I want to edit or close a job posting so that I can keep listings accurate and up to date.
8. As a recruiter, I want to view a list of all applicants for a specific job posting so that I can review candidates efficiently.
9. As a recruiter, I want to update an applicant's status (e.g., move them to Interview or Rejected) so that job seekers are kept informed of their progress.
10. As a recruiter, I want to search and filter candidate profiles by skills or experience so that I can identify strong matches for my open roles.

A couple of things worth flagging for your log: your assignment requires the AI-generated batch to be exactly 10 stories from one clean prompt, kept separate from team-authored ones — so if you're combining these with your existing 5, make sure your submission clearly labels which stories came from AI vs. which your team wrote independently. Also worth double-checking there's no overlap between this AI batch and your team's own brainstormed stories, since the assignment wants those kept distinct.

Database test code:

okay lets test it my code first and if all is good, pls show me the steps to safely merge it to main. Here is all of my code :

```
base.py: from sqlalchemy.orm import declarative_base
Base = declarative_base()
connection.py: import os from dotenv import load_dotenv
from sqlalchemy import create_engine from sqlalchemy.orm import sessionmaker
load_dotenv() DATABASE_URL = os.getenv("DATABASE_URL") if DATABASE_URL
and DATABASE_URL.startswith("postgres://"): DATABASE_URL =
DATABASE_URL.replace("postgres://", "postgresql://", 1) engine =
create_engine(DATABASE_URL) SessionLocal = sessionmaker(autocommit=False,
autoflush=False, bind=engine) def get_db(): """ Creates a temporary database session
for a single API request, and automatically closes it when the request is done. """ db =
SessionLocal() try: yield db finally: db.close() _init_.py : # When we use alembic,
instead of it trying to find the models in the database folder, it will look for them in this
file. from App.models.user import User user.py: from sqlalchemy import Column,
Integer, String, DateTime from datetime import datetime, timezone from
App.database.base import Base # "Base" is basically the interface that allows python to
manipulate the database. It is the base class that all models will inherit from. class
User(Base): """ The User model defines the 'users' table structure in the PostgreSQL
database. It stores credentials required for user registration and login. """
__tablename__ = "users" # Primary key id = Column(Integer, primary_key=True,
index=True) # Registration details username = Column(String(50), unique=True,
index=True, nullable=False) email = Column(String(100), unique=True, index=True,
nullable=False) # Secure login detail (this will be stored as a HASHED password, never
plaintext) hashed_password = Column(String(255), nullable=False) # Audit trail (tracks
when the user registered) created_at = Column(DateTime, default=lambda:
datetime.now(timezone.utc))
```



10 sites

Here are top web results for exploring this topic:



Reddit · <https://www.reddit.com>

How should I go about testing my database connection code? - Reddit

```
... am at # orm.py from sqlalchemy import create_engine from sqlalchemy.orm import sessionmaker from  
sqlalchemy.ext.declarative import declarative_base import os from dotenv import ... orm.py from sql
```



Stack Overflow · <https://stackoverflow.com>

Is there a way to test an SQLAlchemy Connection? - Stack Overflow

7 Answers 7. Sorted by: Reset to default. Highest score (default), Trending (recent votes count more), Date modified (newest first), Date created (oldest first). This answer is useful. 14. This answer



SQLAlchemy · <https://docs.sqlalchemy.org>

Working with Engines and Connections - SQLAlchemy Documentation

The Connection object always emits SQL statements within the context of a transaction block. The first time the Connection.execute() method is called to execute a SQL statement, this transaction is be



GitHub · <https://github.com>

sqlalchemy-orm-tutorial/examples/common/base.py at master - GitHub

```
from sqlalchemy import create_engine from sqlalchemy.ext.declarative import declarative_base from sqlalchemy.orm  
import sessionmaker engine = create_engine('postgresql://dbuser:dbpassword@localhost:54
```



Medium · <https://johncox-38620.medium.com>

Creating a Test Database: PyTest + SQLAlchemy - Medium

Creating a Test Database: PyTest + SQLAlchemy Disclaimer: I'm a total newb with Python and the ecosystem that surrounds it. I come from Rails and apparently I've been completely spoiled by how ...



OneUptime · <https://oneuptime.com>

How to Use SQLAlchemy ORM for Database Operations - OneUptime

SQLAlchemy is Python's most widely used and powerful Object-Relational Mapping (ORM) library. It provides a full suite of well-known enterprise-level persistence patterns while giving you complete fle



YouTube · <https://www.youtube.com>

How to Connect to a Database with SQLAlchemy | Flask and Python ...

In this video I show how to configure SQLAlchemy in a Flask backend. I also show how to create entities which reflect some tables in a database. And show how some relationships are managed with SQLAlc



Google Colab · <https://colab.research.google.com>

SQLAlchemy - Google Colab

SQLAlchemy is the Python SQL toolkit and Object Relational Mapper that gives application developers the full power and flexibility of SQL. We're going to show how to create a database, add some data a



PlanetScale · <https://planetscale.com>

Using MySQL with SQLAlchemy: Hands-on Examples - PlanetScale

SQLAlchemy is a popular Python library that gives you many tools to interact with SQL databases. With SQLAlchemy, you can do things like send raw queries to a database, programmatically construct SQL



DEV Community · <https://dev.to>

Setting up a standalone SQLAlchemy 2.0 ORM application

SQLAlchemy is a widely used database toolkit for Python, providing a SQL abstraction layer covering... Tagged with python, learning, database, sqlalchemy.

show me step by step

Your code is perfectly written, well-commented, and ready to go! Let's get this tested live right now and then walk through the steps to merge it safely.

Part 1: Testing Your Code Live

We will use a quick test script to verify your connection string and tell SQLAlchemy to physically build your 3NF `users` table in your Neon cloud database.

Step 1: Create a test file

In your root project folder (`SOEN341-2026`), create a temporary file named exactly `test_db.py`.

Step 2: Add this testing script into it

Paste this code exactly into your `test_db.py` file:

python

```
import sys
import os

# This tells Python to look inside the current directory for your 'app'
folder
sys.path.append(os.path.abspath(os.path.dirname(__file__)))

from app.database.connection import engine
from app.database.base import Base
from app.models.user import User # This runs your __init__.py and loads
the model

print("🔄 Attempting to connect to your Neon Cloud Database...")
try:
    # This command tells SQLAlchemy to create all tables it finds attached
    to Base
    Base.metadata.create_all(bind=engine)
    print("✅ Success! Your 3NF 'users' table has been created in your
cloud database.")
except Exception as e:
    print("❌ Connection Failed! Error details below:")
    print(e)
```

Use code with caution.

Step 3: Run the test in your terminal

Make sure your virtual environment is active (`.venv`) and run the script:

bash

```
python test_db.py
```

Use code with caution.

- **If it displays  Success!:** Your pipeline works perfectly. Log into your Neon web dashboard and click the **Tables** tab. You will see your new `users` table sitting there live.
- **If it fails:** Double-check your `.env` file to ensure the `DATABASE_URL` link was copied correctly from Neon.

Step 4: Clean up

Once the test passes, you can safely delete the temporary `test_db.py` file so it doesn't clutter your codebase.

Part 2: Safely Committing and Merging to Main

Now that your configuration is verified, let's use Git to safely save your work and merge it into your team's `main` branch.

Step 5: Verify your tracking tree

Run this command to check that Git is tracking the correct files:

```
bash
git status
```

Use code with caution.

Make sure you see your `app/` folder, `requirements.txt`, and `.gitignore` listed. **Crucially, verify that `.env` and `.venv/` are NOT listed.**

Step 6: Stage and commit your files

bash

```
git add app/ requirements.txt .gitignore
git commit -m "Initialize backend layout, set up SQLAlchemy connection,
and add 3NF User model"
```

Use code with caution.

Step 7: Push your feature branch to GitHub

bash

```
git push origin feature/setup-database
```

Use code with caution.

Step 8: Open and merge the Pull Request on GitHub

1. Open your web browser and go to your team's repository page on **GitHub.com**.
 2. Look at the top of the page for a yellow banner that reads "**Compare & pull request**". Click it.
 3. Title your PR clearly (e.g., `Setup Backend Base Architecture and User Model`).
 4. Click the green **Create pull request** button.
 5. If your project group has review policies, notify your teammates to give it an approval stamp.
 6. Click the green **Merge pull request** button, then select **Confirm merge**.
-

Once you complete the merge, your team's backend foundation is officially set up. Let me know:

- Did the connection script return a **success message** or did you run into any **Python import/module errors**?
- Once merged, would you like to move on to setting up **Alembic** so your team can automatically track any future structural changes to this table?

```
(.venv) Safadi@MacBook-Air-5 SOEN341-2026 % python test_db.py Traceback (most recent call last): File "/Users/Safadi/SOEN341-2026/test_db.py", line 7, in <module> from App.database.connection import engine ImportError: cannot import name 'engine' from 'App.database.connection' (/Users/Safadi/SOEN341-2026/App/database/connection.py) (.venv) Safadi@MacBook-Air-5 SOEN341-2026 %
```

The error happens because Python is looking inside your `App/database/connection.py` file, but it cannot find the variable named `engine`.

Looking closely at your files, this is caused by a capitalization mismatch! In your `user.py` file, you imported from `App` (capital A), but the script is looking for `app` (lowercase a), or there might be an issue with how the file saved.

Let's fix this instantly with these two quick steps:

Step 1: Fix the script imports

Update your `test_db.py` file to use the exact uppercase `App` naming convention that matches your folders. Change your script to look exactly like this:

```
python
import sys
import os

# Force Python to look in the root folder
```

```
sys.path.append(os.path.abspath(os.path.dirname(__file__)))

# Use 'App' with a capital A to match your exact file structure
from App.database.connection import engine
from App.database.base import Base
from App.models.user import User

print("🔄 Attempting to connect to your Neon Cloud Database...")
try:
    Base.metadata.create_all(bind=engine)
    print("✅ Success! Your 3NF 'users' table has been created in your cloud database.")
except Exception as e:
    print("❌ Connection Failed! Error details below:")
    print(e)
```

Use code with caution.

Step 2: Verify `connection.py` is saved

Make sure you actually saved the contents inside your `App/database/connection.py` file. If the file is blank or wasn't saved by your text editor, Python won't find `engine` inside it.

Open `App/database/connection.py` and verify it contains this line at the bottom:

```
python
engine = create_engine(DATABASE_URL)
```

Use code with caution.

Once you verify the file is saved and you update `test_db.py`, clear your terminal and run it again:

bash

```
python test_db.py
```

Use code with caution.

Let me know **what the terminal prints out** after running it with this updated script so we can get your green success checkmark!

```
(.venv) Safadi@MacBook-Air-5 SOEN341-2026 % python test_db.py
Traceback (most recent call last):
  File "/Users/Safadi/SOEN341-2026/test_db.py", line 7, in <module>
    from App.database.connection import engine
  File "/Users/Safadi/SOEN341-2026/App/database/connection.py", line 17, in <module>
    engine = create_engine(DATABASE_URL)
  File "<sqlalchemy generated wrapper for sqlalchemy.engine.create.create_engine>", line 2, in create_engine
    return target(fn, url, **kwargs)
  File "/Users/Safadi/SOEN341-2026/.venv/lib/python3.14/site-packages/sqlalchemy/util/deprecations.py", line 281, in warned
    return fn(*args, **kwargs) # type: ignore[no-any-return]
  File "/Users/Safadi/SOEN341-2026/.venv/lib/python3.14/site-packages/sqlalchemy/engine/create.py", line 602, in create_engine
    dbapi = dbapi_meth(**dbapi_args)
  File "/Users/Safadi/SOEN341-2026/.venv/lib/python3.14/site-packages/sqlalchemy/dialects/postgresql/psycopg.py", line 497, in import_dbapi
    import psycopg
ModuleNotFoundError: No module named 'psycopg'
```

Run `pip install "psycopg[binary]"` in your terminal because SQLAlchemy 2.0 tries to use the modern `psycopg` (v3) driver by default, which is missing from your environment.

Why This Happened

By default, when SQLAlchemy sees a `postgresql://` connection string, it looks for the newest PostgreSQL driver package named `psycopg` (version 3). Since you previously installed `psycopg2-binary`, SQLAlchemy didn't find the exact v3 package it expected.

How to Fix It Right Now

Step 1: Install the missing driver

Make sure your virtual environment is active (`.venv`) and run:

```
bash
```

```
pip install "psycopg[binary]"
```

Use code with caution.

Step 2: Update your master requirements list

Keep your team's `requirements.txt` file completely up to date by re-freezing your packages:

```
bash
```

```
pip freeze > requirements.txt
```

Use code with caution.

Step 3: Run your database test again

```
bash
```

```
python test_db.py
```

Use code with caution.

Let me know if you see the  **Success!** message pop up after running this installation command!

it worked

Awesome! Congratulations! 🎉

Your Python code successfully dialed your **Neon cloud database**, traversed the pipeline, and built your 3NF `users` table out on the internet. If you check your Neon dashboard right now under the **Tables** tab, you will see your brand new `users` schema sitting there perfectly.

Now that the backend plumbing is verified and working, it is time to perform a **clean, safe merge to your `main` branch** so your teammates can pull down your foundational code.

Follow these exact steps to wrap up this branch:

Step 1: Delete the temporary test script

Since `test_db.py` was just a scratchpad to verify the infrastructure connection, delete it from your folder so it doesn't get pushed to GitHub:

```
bash
```

```
rm test_db.py
```

Use code with caution.

Step 2: Verify your Git tree

Run this command to check exactly what files Git is about to save:

```
bash
```

```
git status
```

Use code with caution.

- Under **"Untracked files"**, you should see your `App/` folder, `requirements.txt`, and your updated `.gitignore`.
- **Crucially, make sure you do NOT see `.env` or `.venv/` listed.** (Your `.gitignore` file is working perfectly to hide them).

Step 3: Stage and commit your backend files

Run these commands to save your verified architecture to your local branch:

```
bash
```

```
git add App/ requirements.txt .gitignore
```

```
git commit -m "Initialize backend layout, configure SQLAlchemy, and deploy 3NF User model"
```

Use code with caution.

Step 4: Push the branch up to GitHub

Upload your branch from your laptop up to the online GitHub repository:

bash

```
git push origin feature/setup-database
```

Use code with caution.

Step 5: Merge it into Main on GitHub

1. Open your web browser and navigate to your team's repository on **GitHub.com**.
 2. At the top of your repository page, look for a yellow banner that reads "**Compare & pull request**". Click that button.
 3. Give your pull request a clean title (e.g., `Setup Base Backend Architecture and User Database Model`).
 4. Click the green **Create pull request** button.
 5. Once your team members look it over, click the green **Merge pull request** button, followed by **Confirm merge**.
-

What's next for your SOEN 341 team?

Your master architecture is now safely on `main`. You can tell your teammates that the core infrastructure is live!

They can now run **Step 1 through Step 5** from the documentation you wrote earlier: they will pull `main`, spin up their local `.venv`, run `pip install -r requirements.txt`, plug in the shared Neon link into their local `.env` files, and they will instantly be ready to code alongside you.

How are you feeling about the setup now? Would you like to move on to setting up your **FastAPI endpoints** for the registration and login logic next, or do you want to take a break here?